

lhstya - Team17

lhstya - Team17 (*student proposal*)

Assessment: ♦ Fit with adjustments · **Category:** Games · **Assessed:** 2026-09-22 · **Status:** awaiting instructor approval

Proposal (as submitted): *lhstya* is a **mobile lane-based deckbuilder defense strategy RPG** (in the style of **Arknights**, **Clash Royale** and **Counter:Side**) where you enter battle with a fixed deck of **Units and Artifice** to fight off waves of enemies — using resource management to strategically summon heroes/soldiers and use artificery to defend your territory. Named load-bearing C++: a **C++ Entity Component System** decoupling data from execution to minimise memory fragmentation + pointer chasing; and a **C# scripting layer** for fast iteration + hot-reloadable data on top of the C++ native core (which handles the heavy logic). Named technical features: an **entity logic state machine** (moving/conflict); **special rendering with entity culling**; the **ECS with C# scripting + hot-reloadable data**; and an **entity cone-detection system** (field-of-view). Tools: a **custom GUI editor** (like Unity/Godot/Unreal) for debugging, editing + viewing data/scenes and live test-running UI + gameplay; and **data serialization** (JSON read optimally into the sim loop). Design features: **Units** (Named + Soldiers) with distinct attack patterns + skills/ultimates that interact dynamically (e.g. auto-counterattack when hit, or a teleport-behind-vision skill); **lore-linked team synergies** (units related by lore gain effects together — linked ultimates, stacking buffs by count on field); **two decks with different functionality** (an **Artifice/Magic** deck on cooldown where every card is always accessible, and a **Summon/Unit** deck on a shuffle where used units cycle back into the shuffle); and a **dynamic map** that magic + units interact with (spells summon walls that block pathing forcing detours, or build a temporary portal/bridge to bypass terrain). Team: **6 CS + 2 Designers + 1 Artist**.

Adjustments required (why ♦):

1. **Monetization — the "gacha" model must be non-pay-to-win.** The module's clean-monetization rule forbids pay-to-win: real-money purchases must be **cosmetic-only**. A gacha that sells gameplay-

affecting **Units** for real money is pay-to-win and is not permitted. Keep unit acquisition **earned by play** (in-game currency only), or restrict any real-money purchases to cosmetics — and design this explicitly (see the M2 Monetization & Scope-Integrity design goal). This is a design-model change, not a tech blocker.

2. **The C# scripting layer is an ambitious, non-required extra — keep the C++ core load-bearing.** The module is built around a **C++ core + data-driven JSON hot-reload** (which already gives you fast iteration without recompiling). Embedding a C# scripting runtime (Mono/.NET) on top adds real toolchain + interop complexity and is **not required**. Treat C# scripting as a **stretch**: deliver the required computational logic in C++ with JSON hot-reload first; add a scripting layer only if time allows, and ensure the graded C++ engine carries the load-bearing logic regardless.

The core game (lane deckbuilder defense, ECS, two-deck systems, dynamic map) is a strong, well-scoped Games fit once these two are settled.

Note (module completeness): the app needs an **External Database** (unit rosters / collection / accounts) — use Firebase and/or the team's Linux server (set the seam up + basic-test it by M2). *Platform:* Arknights/Clash-Royale-style → a **mobile** build (Native Android, Pixel 9a).

References:

- [Arknights — Arknights Wiki](#) (the named benchmark: a **gacha tower-defense** where you deploy collectible Operator units onto a grid to stop waves along fixed lanes, with unit skills that zone/pull/push enemies — the deploy-units-defend-lanes + collection precedent)
- [Clash Royale — Wikipedia](#) (the named benchmark: a real-time **lane card/deck** game — collect cards, build a deck, and deploy troops on a cooldown/elixir economy to defend and attack lanes — the deck + resource-economy + lane precedent)
- [Counter:Side — Gacha Games Wiki](#) (the named benchmark: a side-scroll "Line Battle" deckbuilder gacha where you deploy up to eight units in real time on a **deployment-point cost** economy — the deckbuilder + deployment-cost + auto-battle-line precedent)

Core Tech Requirements

Legend: ✓ Applies directly | ♦ Applies with domain adaptation | ● Optional | ✗ Not needed | » Second trimester

Core Tech Requirement	Ihstya
External Database	✓
Authentication »	●
Analytics »	●
Real-time C++ Simulation Loop and/or C++ Performant Service	✓
Application Communication	✓
Continuous Integration	✓
Data-Driven Architecture	✓
Front-end + Back-end	✓
Custom Tools Development	✓
2D Graphical Output	✓
3D (strong students only)	✗
Networking System	●

How each requirement applies:

Platform fit: **Main:** Native mobile (Native Android — C++ engine via JNI rendering through OpenGL ES in a GLSurfaceView, Kotlin app shell; device: Pixel 9a) — a touch-first lane deckbuilder. The Native PC build is the dev/tools vehicle (the custom GUI editor). **Extended:** Firebase + a Linux server the team sets up by M2 (full use T2) for accounts + unit-collection persistence.

Front end: A 2D lane-battle renderer — the many units (heroes/soldiers) with culling, the two decks (cooldown Artifice + shuffle Summon), skill/ultimate VFX, the dynamic map (walls/portals/bridges), and the resource/HUD.
Languages: Native — Kotlin Native Android UI + the C++ engine via JNI

(Android Studio); OpenGL ES for the realtime 2D in a GLSurfaceView; some screens (collection, deck-build, menus) can be pure Native UI. Web/desktop — SDL2/3 + OpenGL ES for the PC dev/tools build.

Back end: The real-time C++ engine — the ECS (contiguous storage, low fragmentation), the entity state machine (moving/conflict), unit skills/ultimates + lore synergies, the two-deck economy (cooldown + shuffle), the entity cone-detection, the dynamic-map pathing, and special rendering with culling — driven by JSON hot-reload (and, as a stretch, a C# scripting layer); plus Firebase for accounts/collection. **Languages:** C++ (native core — load-bearing); optional C# scripting layer; Firebase on the Extended tier.

Backend scope: Local now, server-ready (T2) — the accounts/collection seam set up & basic-tested by M2, full use in T2.

- *Loop and/or service:* the C++ ECS loop is the heart — it runs the entity state machine, unit skills/ultimates + synergy resolution, the two-deck economy, cone-detection and the dynamic-map pathing at a stable step, with data hot-reloaded from JSON (the C++ core carries the heavy logic; C# scripting is an optional layer, not the graded core).
- *Data-driven:* units (heroes/soldiers) + skills/ultimates, lore-synergy rules, the two decks, artifice/magic effects and the dynamic-map spells are all data — a new unit, skill or spell is a data change (authored in the editor, JSON-serialized).
- *2D:* the lane-battle renderer with entity culling, skill/ultimate VFX and the dynamic map (custom-rendered). *3D:* not needed.
- *Custom tool:* the **custom GUI editor** (Unity/Godot/Unreal-like) — debug, edit + view data/scenes, and live test-run UI + gameplay — plus the JSON serialization pipeline.
- *Networking:* single-player PvE lane defense as described (no real-time PvP), so a Networking System is optional (●). *External DB:* accounts + unit collection (Firebase — add per the module). *Auth » / Analytics »:* module-deferred to T2 (●).

Suggested Tech Goals

G1–G3 ★ are the common goals (fixed for all categories); the other 3 are picked from the Games pool in [milestone-goals-pool-per-category.pdf](#) (G1–G3 ★ common, G4+ picks).

M1:

- **G1 ★ Application Shell, Real-time C++ Simulation Loop, Input & Core Logic** — the ECS loop + the entity state machine + serialization.

Tech & dependencies

- **Tech: Web build:** SDL2/3 window + a fixed-timestep C++ loop over an ECS (contiguous storage, low fragmentation) running the entity state machine (moving/conflict), with JSON data serialization read efficiently into the loop; C++17/20, CMake, Git. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
 - **Prerequisites:** None — foundation of this project.
 - **Feeds into:** Every other goal; rendering (M1G4), AI/cone-detection (M1G8) and the dynamic map (M1G14) run on its tick.
 - **G2 ★ Data-Driven Architecture of Core Objects / Entity System** — the ECS + JSON hot-reload (C# scripting optional, see adjustments).
- Tech & dependencies
- **Tech:** The ECS data model with units + skills/ultimates, lore-synergy rules, the two decks and dynamic-map spells in hot-reloadable JSON — a new unit/skill/spell is a data change. (A C# scripting layer is a stretch that sits on top; the C++ core remains the load-bearing, graded engine.)
 - **Prerequisites:** G1 (loop & entity hooks).
 - **Feeds into:** M2G2 (the GUI editor), M2G13 (deck/skill systems), M1G8 (AI data).
 - **G3 ★ Debugging, Profiling, Stress Test & CI**
- Tech & dependencies
- **Tech:** ImGui overlays (entity states, cone-detection, deck/resource state), Tracy profiling of a wave with many units + culling, Catch2 tests

(state machine, synergy resolution, deck cycling, data loading), on a GitHub Actions matrix — the base of the GUI editor's debug views.

- **Prerequisites:** G1 (loop to instrument).
- **Feeds into:** M2G2 (the editor); M2G3 (CI hardening).

- **G4 2D Graphics** — the lane-battle renderer + **special rendering with entity culling**.

Tech & dependencies

- **Tech: Web build:** A sprite renderer with entity culling (only draw on-screen/relevant entities) for the many units, skill/ultimate VFX, the two decks and the dynamic map, plus the resource HUD; SDL2/3/OpenGL ES, GLSL, stb_image, FreeType. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G1 (loop & window).
- **Feeds into:** M2 skill/ultimate VFX polish; the editor's scene view shares this renderer.

- **G8 Enemy AI & Pathfinding** — the entity state machine behaviours + **cone detection** + dynamic-map pathing.

Tech & dependencies

- **Tech:** A* / flow-field pathfinding along the lanes that re-paths around summoned walls (detours) or through portals/bridges, an entity cone-detection (field-of-view) system feeding unit decisions, and the moving/conflict state machine; custom C++, deterministic.
- **Prerequisites:** G1 (tick), G2 (unit data), G14 (dynamic map).
- **Feeds into:** M2G10 (advanced AI — skill/ultimate behaviours); the teleport/counter/vision mechanics.

- **G14 Level / Tile Map System** — the **dynamic map** (lanes + walls/portals/bridges affecting pathing).

Tech & dependencies

- **Tech:** A lane/tile map (JSON) with collision + pathing data that spells mutate at runtime — summon walls that block (forcing detours) or portals/bridges that bypass terrain — with the pathfinding querying the live graph; spatial indexing.
- **Prerequisites:** M1G2 (map/spell data), G4 (rendering).
- **Feeds into:** M2G2 (the editor authors maps); the dynamic-map design pillar.

M2:

- **G1 ★ Architecture — Optimization & Platform Validation** — the ECS/culling perf + the persistence seam.

Tech & dependencies

- **Tech: Web build:** Cache-friendly ECS iteration, entity-culling tuning, memory pools, Tracy profiling of a heavy wave, ASan on MSVC, and on-device validation (Pixel 9a); the Firebase/Linux-server seam (accounts/collection) set up (account + software + basic connectivity test) by M2. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** All M1 goals (hardens the M1 codebase).
- **Feeds into:** The Native-Android deliverable + a stable heavy-wave battle.

- **G2 ★ Content Editor — Core (Custom GUI Editor)** — the Unity/Godot/Unreal-like editor.

Tech & dependencies

- **Tech:** Dear ImGui + the shared renderer: debug + view + edit data/scenes (units, skills, decks, maps), with round-trip JSON serialization, live preview and undo/redo — the base of the "live test-run" editor.
- **Libraries/Software:** Dear ImGui, nlohmann/json, the project's SDL2/3/OpenGL ES renderer (preview).
- **Prerequisites:** G2 (data model), M1G3 (debug overlays).
- **Feeds into:** M2G6 (live test-running UI + gameplay); all content.

- **G3 ★ CI — CMake, Code Quality, Build Standards & Packaging**

Tech & dependencies

- **Tech: Web build:** CMake presets, clang-format + clang-tidy gates, reproducible Windows + Android builds, tests (state machine, synergy, deck cycling, dynamic-map pathing, data loading) and packaging via GitHub Actions. **Native build:** Android Studio (Kotlin Native Android UI + the C++ engine via JNI); OpenGL ES drawn in a GLSurfaceView where realtime 2D is needed — no SDL on native, some screens are pure Native UI.
- **Prerequisites:** G3 (CI & test foundation).
- **Feeds into:** Release/showcase builds of every goal.

- **G6 Content Editor — Advanced Tool Features (Live Test-Run UI + Gameplay)** — the GUI editor matured.

Tech & dependencies

- **Tech:** Dear ImGui advanced widgets: drag-drop scene/unit/map building, live binding of edits into a running session, and a play/test mode running the engine's UI + gameplay in-editor; deep undo/redo. (If a C# scripting layer is pursued, this is where hot-reload of scripts would surface — stretch.)
- **Prerequisites:** M2G2 (editor core).
- **Feeds into:** Fast content authoring; the custom-tools showcase.

- **G10 AI — Advanced (Unit Skills/Ultimates & Cone-Detection Behaviours)** — the dynamic unit interactions.

Tech & dependencies

- **Tech:** Behaviour trees / FSMs for the skill/ultimate behaviours (auto-counterattack on being hit, teleport-behind-vision + slip control, linked ultimates), matured cone-detection decisions, and enemy-wave AI, tuned from data.
- **Prerequisites:** M1G8 (state machine + cone detection).
- **Feeds into:** The dynamic unit-interaction design pillar; the showcase depth.

- **G13 Gameplay Systems II** — the **two-deck economy + lore synergies + resource management**.

Tech & dependencies

- **Tech:** C++ systems over the ECS: the Artifice deck (cooldown, all cards accessible) + the Summon deck (shuffle, used units cycle back), the resource-management economy, and the lore-synergy system (stacking buffs by count on field, linked/enhanced ultimates); balancing-data-driven.
- **Prerequisites:** M1G2 (unit/deck data), M1G1 (loop/economy hooks).
- **Feeds into:** The core strategy depth at showcase; T2 unit/deck expansion.

Track Goals & Team Composition

Recommended composition: 6 CS + 2 Design + 1 Art (9) — **as proposed**, and sensible for a **systems-deep** game: two designers can split the heavy design load (the two-deck economy + synergies, the unit/skill/ultimate design, the dynamic-map + wave design, the balance, and — importantly — the **monetization/scope-integrity** design the ♦ adjustment requires), while one artist carries the visual surface. Note this is at the **D + A ≤ 3 cap** (2 Design + 1 Art), so a 2nd artist can't be added without dropping a designer. *Art-scope caution:* a **collection/gacha-style RPG is art-heavy** (many Named units, skills/ultimates VFX) but has only **one artist** — scope the **Named-unit roster small** (a handful of distinct heroes + reusable soldier/enemy art), lean on a strong reusable style, and let the ambitious roster be a T2 stretch. *Scope note (advisory):* a C++ ECS + entity state machine + cone-detection + dynamic-map pathing + the two-deck systems + a Unity-like editor is a **large, ambitious CS scope** even before the C# scripting layer — the ECS lane battle with the two decks, a few units with skills, the dynamic-map walls, and the editor's core is the M1/M2 floor; the C# scripting layer, the full synergy/linked-ultimate web, cone-detection-driven AI and the live test-run editor are the stretch. Settle the **monetization model early** (it changes the collection/economy design) and keep the **C++ core load-bearing** (C# is optional).

Track goals — suggested Design / UI-UX / Art / TEAM picks for this project; deliverables & quality ladders live in the track pool documents. Full load = 4 goals per designer per milestone across Design + UI/UX (8 combined for 2 designers).

- **Design M1:**

- M1D1 ★ Features Design Document (the lane-deckbuilder-defense USP, the two-deck system, lore synergies, the dynamic map and the collection model — with the monetization model stated as non-pay-to-win; primary benchmark e.g. Arknights / Clash Royale / Counter:Side, plus a Singapore benchmark)
- M1D2 Core Gameplay Loop Prototype (a greybox proving deploy → resource-manage → skill/ultimate → defend with the two decks is fun) · prototype in Unity

- M1D6 Systems & Economy Design (the two-deck economy — cooldown Artifice + shuffle Summon — the resource management and the lore-synergy interlocks)
- M1D8 Puzzle & Mechanic Design (the unit skill/ultimate interactions — auto-counter, teleport-behind-vision, linked ultimates — and the dynamic-map spell mechanics)
- M1D5 Level & Progression Design (the dynamic maps, the enemy waves + difficulty curve, and how walls/portals reshape encounters)

• **Design M2:**

- M2D1 ★ Features Design Iteration & Showcase Readiness
- M2D2 Working Prototype — Full-Content Build (the polished lane battle with the two decks + a few synergised units, in the team's engine + editor)
- M2D3 Systems Balance & Tuning — Data-Driven (unit/deck/synergy/wave balance via the editor's JSON, no code changes — critical for a deckbuilder)
- M2D8 **Monetization & Scope-Integrity Design** (the ♦ adjustment: design a clean, non-pay-to-win model — cosmetics and/or earn-by-play collection, NO real-money gameplay power — plus a scope-integrity statement)
- M2D5 Playtesting & Iteration Program (deckbuilders live or die on balance — structured playtests with before/after changes)
- M2D7 Narrative Presentation & Delivery (the lore that drives the team-synergy system — unit/faction identities and why lore-linked units combine — delivered in-game)

• **UI/UX M1:**

- M1U1 ★ UI Standards & Wireframe Baseline — standards for the deck/hand/resource HUD, the collection/deck-build screens and the battlefield, plus a wireframe baseline.
- M1U2 UX Flows & Wireframes — Menus → HUD (the collection → deck-build → battle → results flow mapped before pixels get expensive).
- M1U3 Accessibility & Readability Foundations (legible unit/skill iconography and touch targets on a busy lane).

- **UI/UX M2:**

- M2U1 ★ Functional UI & Usability — the collection, deck-build, battle HUD and results navigable unaided on the Native-Android target.
- M2U2 ★ UX Validation & Final Polished UI — validated, polished deploy/skill/deck UX tested with players on the Pixel 9a.

- **Art M1:** 1 artist (Named units + soldiers/enemies + skill VFX + dynamic map + UI — scope the roster small)

- M1A1 ★ Art Bible / Proof of Concept (the world's visual direction — tone, palette, unit-identity language)
- M1A2 Character Concepts & Model Sheets (a scoped set of Named units + reusable soldier/enemy designs)
- M1A3 Environment, Props & VFX Concepts (the dynamic map — walls/portals/bridges — and the skill/ultimate VFX concepts)

- **Art M2:**

- M2A1 ★ Critical Asset Production & In-Engine Visual Quality
- M2A2 Character Animation — Extensive (unit attack patterns, skills, ultimates + the linked-ultimate moments)
- M2A5 VFX — Critical Effects & Transitions (the skills/ultimates, linked-ultimate flourishes, and the dynamic-map spell effects)

- **TEAM M1:**

- M1TA1 ★ Overall Audio Experience — initial BGM/SFX from the week-5 needs list present & working in the prototype (battle BGM, summon, magic-cast, skill/ultimate, hit and wave cues), suitable and correctly triggered.
- M1TA2 ★ Audio Engine Functionality — data-driven audio pipeline (add/replace files without recompiling) + reliable multi-channel playback for a busy lane.

- **TEAM M2:**

- M2TA1 ★ Overall Audio Experience — standard Games floor (battle BGM + summon / magic / skill / ultimate / hit / victory SFX, ≥6 types), scaling with lane intensity + linked-ultimate moments.
- M2TA2 ★ Audio Engine Functionality — audio properties de/serialised and the full playback functionality the build needs (multi-channel mixing; distinct ultimate/linked-ultimate cues).

- M2TP1 ★ Final Presentation — 7 min in front of class +
instructors: live custom-tools showcase (the custom GUI editor),
prototype demo, team post-mortem (wk14)
-

*Generated by the Project Proposal Assessor skill · grounding: live folder
(Project 2 Technical adjustments/) · references verified 2026-09-22.*